

Plain2MeTTa v2 — Specification & Implementation Plan

Author: ProtomegaTron (on behalf of Ben Goertzel) **Date:** 2026-08-06 **Status:** DRAFT
— awaiting review by ProtoCosmoBot, Hugo, and Ben before implementation

1. Executive Summary

Plain2MeTTa currently takes sketchy `.plain` specs and compiles them into structured validation artifacts (SpecAtom-HS JSON, PeTTa reified atoms, diagnostics reports). It excels at source-preserving indexing, conservative concept extraction, and fail-closed validation obligations. **It does not generate executable code, and it does not elaborate incomplete specs into detailed ones.**

The current system's value is real — source provenance, validation scaffolding, and conservative concept handling are solid foundations. But its output is pre-executable: it tells you what's underspecified, not what to build.

What's needed: a staged pipeline that takes a sketchy spec all the way to reviewed, executable, tested code, while preserving a distinct non-executable logical representation for formal review:

1. **Elaborate** the sketch into a detailed English-language spec + test spec (LLM-assisted, human-reviewable)
2. **Build and review a literate logical IR / verification skeleton** — a logically explicit but non-executable MeTTa theory, with declared operational holes (LLM-assisted, reviewable by humans and formal reasoners)
3. **Compile** the reviewed skeleton/spec into literate MeTTa code + grounded Python modules + executable tests (LLM-assisted)
4. **Execute** the tests and display results in the web UI

The existing codebase provides the scaffolding for phase 1 (source indexing, concept extraction, validation obligations). The new work is the LLM-assisted elaboration and compilation, the review workflow, and the test execution infrastructure.

2. What the System Currently Does

2.1 Input Format

`.plain` files with explicit section markers and lightweight syntax:

```
***definitions***
- :PriceObservation: is an observed market price at a timestamp.

***functional specifications***
- [id:PRED-1] The experiment trains a model to predict :ReturnLabel24h: from price features.

***acceptance tests***
- [covers:PRED-1] Given a known dataset, when the model is trained and evaluated, then MSE is finite.
```

2.2 Current Processing Pipeline

1. **Source indexing** — stable IDs, SHA-256 digests, exact byte/line spans
2. **Concept extraction** — conservative `:Concept:` markers, external references, definition/glossary bullets
3. **Validation obligations** — fail-closed checks for source provenance, fact arity, semantic objects, requirement coverage, ML methodology, security/privacy
4. **PeTTa reified atoms** — structured atoms for source manifest, objects, facts, validation records; explicitly refuses executable skeleton generation
5. **Diagnostics** — human-readable validation report with pass/fail/unknown counts and blocking questions

2.3 Current Web UI

- Left panel: text editor for `.plain` input, example loader dropdown, Compile button
- Right panel: three tabs (Diagnostics, MeTTa Atoms, JSON), summary badges (pass/fail/unknown/refusals)
- Single API endpoint: `POST /api/compile` → instant deterministic compilation

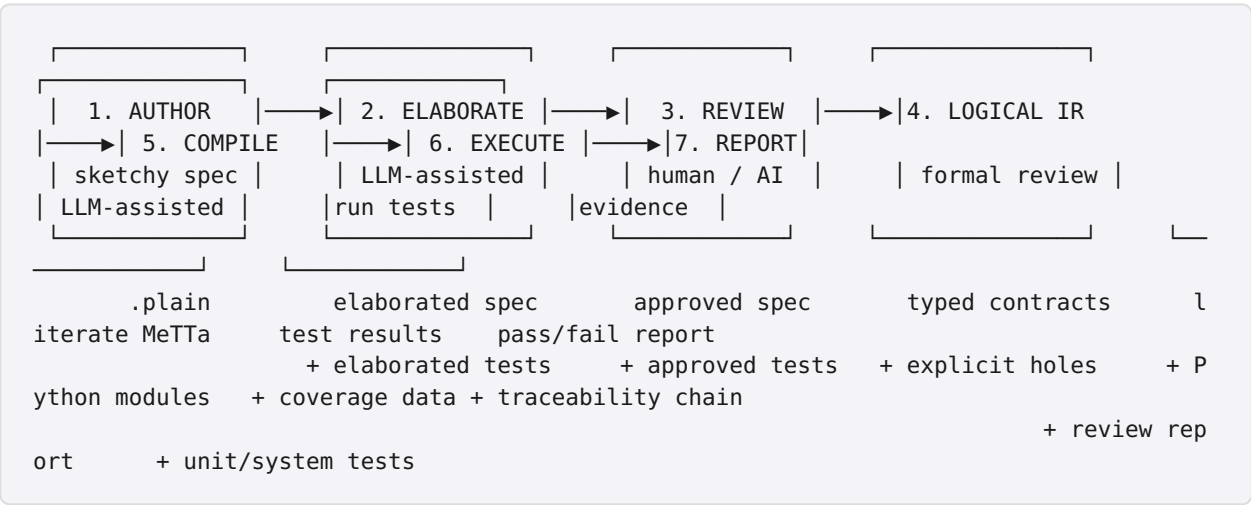
2.4 What It Does Not Do

- Does not elaborate incomplete specs into detailed ones
- Does not generate executable code (explicitly refused)
- Does not invoke LLMs at any stage
- Does not run tests or display test results

- Has no review/approval workflow
- Has no concept of “guidance prompts” to steer elaboration or compilation
- Cannot produce literate programming output
- Has no versioning of spec → code → test-result chains

3. What the System Should Do

3.1 The Full Pipeline



3.2 Phase 1: Author (unchanged)

The user writes a sketchy `.plain` spec. The current input format is retained. Optionally, the user provides a **guidance prompt** — free-text instructions that steer the elaboration (e.g., “This is for a financial analytics pipeline; use pandas and scikit-learn; target Python 3.10+”).

3.3 Phase 2: Elaborate (NEW — LLM-assisted)

Input: sketchy `.plain` spec + optional guidance prompt

Process: An LLM (configurable — could be Claude, GPT, a local model via OpenClaw gateway) takes the sketchy spec and produces:

1. **Elaborated functional spec** — detailed, reviewable English prose. Every vague bullet is expanded into precise behavior descriptions. Ambiguities are resolved or flagged as explicit questions. Each spec item retains its `[id:...]` tag and `:Concept:` references.

2. **Elaborated test spec** — detailed, reviewable English prose describing how to verify each functional requirement. Each constraint in the spec becomes at least one test. Test specs use the format:

```
***system tests***  
- [covers:PRED-1] TEST-1: Run the full pipeline on a synthetic dataset of 1000  
  daily price observations with a known linear trend. Verify that:  
  (a) The model produces predictions for all test-split timestamps.  
  (b) MSE on the test split is reported and is finite.  
  (c) Directional accuracy exceeds 50% (the trend is learnable).  
  
- TEST-2: Construct a dataset where the only predictive signal is in future  
  (post-prediction-timestamp) data. Run the pipeline. Verify that directional  
  accuracy on the test split does not significantly exceed 50% (no future leakage).
```

Key constraints: - The elaborated spec must be in reviewable English — no code, no MeTTa, no JSON. A domain expert who cannot program should be able to read it, understand it, and identify errors. - The elaborated spec preserves all `[id:...]`, `[covers:...]`, and `:Concept:` markers from the original. - New spec items introduced by elaboration get new IDs. - The LLM must not fabricate external dependencies or assume specific libraries unless the guidance prompt names them. - Elaboration runs through the existing SpecAtom-HS validation pipeline after LLM expansion, so source provenance, concept consistency, and coverage obligations are checked on the elaborated output.

Output artifacts: - `<name>.elaborated.plain` — the elaborated functional spec - `<name>.tests.plain` — the elaborated test spec - `<name>.elaboration-log.json` — LLM interaction trace (prompt, response, model, tokens, timestamp)

3.4 Phase 3: Review (NEW — human or AI committee)

Input: elaborated spec + elaborated tests + validation diagnostics from Phase 2

Process: A human (or an AI advisory committee, or both) reviews the elaborated outputs. The review interface supports:

- Inline editing of the elaborated spec and test spec
- Approve / Request Changes / Reject per section or per item
- Annotation comments
- Re-running elaboration on specific sections with modified guidance

Key constraints: - No compilation happens until the elaborated spec is explicitly approved. - Review status is persisted — the system knows which version of the elaborated spec was approved. - If the original `.plain` spec changes, the elaboration is invalidated and must be re-run.

Output artifacts: - `<name>.elaborated.reviewed.plain` — the approved elaborated spec (may be identical to the elaborated version if no changes were made) - `<name>.tests.reviewed.plain` — the approved test spec - `<name>.review-log.json` — review decisions, comments, timestamps, reviewer identities

3.5 Phase 4: Literate Logical IR / Verification Skeleton (NEW — LLM-assisted)

Purpose: make the approved elaborated specification available as a coherent logical theory for review before implementation choices obscure specification errors. This phase is not application execution. It must never claim to load data, normalize inputs, train models, or perform any other grounded operation.

Input: approved elaborated spec + approved elaborated system-test spec + optional logical-review guidance.

Output: a modular, extensively commented MeTTa document containing:

1. Typed declarations for every material concept and relation.
2. Function signatures/contracts, preconditions, postconditions, invariants, data-flow and ordering constraints.
3. Requirement-to-test obligations and dependency declarations.
4. Explicit typed holes for every unresolved implementation or grounding choice, e.g. `(: fit-scaler (-> TrainingRows Scaler))` together with `(OperationalHole fit-scaler "grounded implementation required")`.
5. Per-section provenance comments linking elaborated-spec IDs, skeleton atoms, later implementation locations, and planned tests.

Acceptance gate: generate a machine-readable review report and block executable compilation for critical findings: missing definitions, inconsistent types, contradictory invariants, uncovered requirements, invalid ordering/data-flow, possible leakage, unreachable obligations, or unmarked operational gaps. A human or advisory-AI committee may repair or approve the skeleton; all findings and decisions are retained.

Terminology guardrail: call this artifact a literate logical IR / verification skeleton, not executable code. Declarative fragments may also be executable MeTTa, but every operational gap must remain explicit.

Output artifacts: - `<name>.logical-skeleton.metta` — literate logical IR - `<name>.logical-review.json` — consistency/completeness findings and dispositions - `<name>.logical-review-log.json` — model/reviewer provenance

3.6 Phase 5: Compile (NEW — LLM-assisted)

Input: approved elaborated spec + approved test spec + approved logical IR + optional compilation guidance prompt

Process: An LLM translates the approved English-language specs into executable artifacts:

1. **Literate MeTTa code** — MeTTa source where each function/type/rule is preceded by the spec item it implements, with `[id:...]` tags preserved as comments. The MeTTa is the primary executable representation.
2. **Grounded Python modules** — where the spec requires computation that is more naturally expressed in Python (data processing, ML model training, I/O, numerical work), the compiler produces Python modules that are grounded into the MeTTa space. The MeTTa calls the Python; the Python does not bypass the MeTTa.
3. **Executable unit tests** — one or more test per spec item, derived from the approved test spec. Each test is tagged with the `[covers:...]` ID it validates.
4. **Executable system tests** — integration/end-to-end tests derived from the `***system tests***` section of the approved test spec.

Key constraints: - The compiled code must implement exactly the approved spec — no features added, no requirements dropped. - Each `[id:...]` requirement must map to at least one test. - The compilation must be traceable: for any line of generated code, you can identify the spec item that motivated it. - The compiler should produce a `<name>.traceability.json` mapping spec IDs → code locations → test IDs. - The LLM compilation runs through the existing SpecAtom-HS validation pipeline afterward — the generated MeTTa is parsed and validated for concept consistency, requirement coverage, and fact-arity correctness.

Output artifacts: - `<name>.metta` — literate MeTTa source - `<name>/` — directory of grounded Python modules (if needed) - `<name>.tests.metta` — MeTTa test harness - `<name>.tests.py` — Python test runner (pytest-compatible) - `<name>.compilation-log.json` — LLM interaction trace - `<name>.traceability.json` — spec → code → test mapping

3.7 Phase 6: Execute Tests (NEW)

Input: compiled code + test suite

Process: The system runs the test suite in a sandboxed environment and captures results.

Output artifacts: - `<name>.test-results.json` — structured results: per-test pass/fail/error/skip, stdout/stderr, duration, coverage data - `<name>.test-results.txt` — human-readable summary

3.8 Phase 7: Report (NEW)

Input: test results + traceability data + all prior artifacts

Process: The system generates a traceability report showing: - Which spec items are covered by passing tests - Which spec items have failing tests (with failure details) - Which spec items lack test coverage - The full provenance chain: original spec → elaborated spec → approved spec → compiled code → test → result

4. Web UI Modifications

4.1 Current UI (retained, enhanced)

The existing left-panel editor and right-panel output tabs remain. They become Phase 1/2 of the workflow.

4.2 New UI Components

4.2.1 Workflow Progress Bar

A horizontal step indicator at the top of the page showing the current phase:

[Author] → [Elaborate] → [Review] → [Logical IR] → [Compile] → [Test] → [Report]
✓ ● ○ ○ ○ ○ ○

Active phase highlighted. Completed phases show checkmarks. Users can click back to any completed phase to inspect its artifacts.

4.2.2 Guidance Prompt Panel

Below the editor, a collapsible text area for the guidance prompt. Separate guidance prompts for elaboration and compilation. Pre-populated with sensible defaults based on the spec content (e.g., if the spec mentions ML, the default guidance includes “use scikit-learn and pandas”).

4.2.3 Elaboration Output Panel

After elaboration, the right panel gains two new tabs: - **Elaborated Spec** — the expanded functional specification, rendered as formatted text - **Test Spec** — the elaborated test specification

These tabs show the LLM-generated output alongside the validation diagnostics. The existing Diagnostics/MeTTa/JSON tabs show the validated version of the elaborated output.

4.2.4 Review Interface

When the user clicks “Review,” the elaborated spec and test spec become editable in the right panel. Each section/item has: - Approve / Reject / Edit controls - A comment field for annotations - A “Re-elaborate this section” button that sends just that section back to the LLM with additional guidance

A “Approve All” button marks the entire elaborated spec as reviewed.

4.2.5 Compilation Output Panel

After compilation, new tabs appear: - **MeTTa Code** — the literate MeTTa source with syntax highlighting - **Python Modules** — any grounded Python code, with syntax highlighting - **Traceability** — interactive table mapping spec items → code → tests

4.2.6 Logical IR Review Panel

After elaborated-spec approval and before compilation, show: - **Logical IR** — syntax-highlighted, literate MeTTa skeleton organized by definitions, contracts, invariants, data flow, test obligations, and operational holes - **Logical Review** — machine-readable findings with severity, linked source clauses, and explicit approve/repair/defer actions - **Grounding Gaps** — a filtered list of every operation that still requires a MeTTa implementation or Python-grounded module

The Compile button remains disabled until critical logical-review findings are resolved or explicitly waived with reviewer identity and rationale.

4.2.7 Test Results Panel

After test execution, a new tab appears: - **Test Results** — summary view (12/14 passed, 2 failed) with: - Color-coded per-test results (green pass, red fail, yellow skip) - Drill-down to individual test output (stdout, stderr, assertion details) - “Re-run Tests” button - Filter by spec item ID - Coverage visualization: which spec items are covered by passing tests vs. failing tests vs. untested

4.2.8 Version History Sidebar (optional, Phase 2)

A collapsible sidebar showing the version history of the current spec project: - Each elaboration, review, compilation, and test run is a versioned snapshot - Users can diff between versions - Users can roll back to a previous version

4.3 API Additions

Endpoint	Method	Description
POST /api/elaborate	POST	Elaborate a sketchy spec (accepts <code>text</code> + <code>guidance</code>)
GET /api/elaborate/:id	GET	Poll elaboration status (may be async for large specs)
POST /api/review/:id	POST	Submit review decisions for an elaborated spec
POST /api/logical-ir/:id	POST	Generate the logical IR from an approved elaborated spec
POST /api/logical-review/:id	POST	Submit or retrieve logical-IR review findings and decisions
POST /api/compile/:id	POST	Compile an approved spec (accepts optional <code>guidance</code>)
GET /api/compile/:id	GET	Poll compilation status
POST /api/test/:id	POST	Run tests for a compiled spec
GET /api/test/:id	GET	Poll test execution status / retrieve results
GET /api/trace/:id	GET	Get the full traceability chain for a spec project
GET /api/versions/:id	GET	Get version history for a spec project

The existing `POST /api/compile` (instant deterministic compilation) is retained as `POST /api/validate` — the pure validation path without LLM involvement.

5. Architecture Decisions

5.1 LLM Integration

The LLM is accessed via a configurable backend — not hardcoded to any specific provider. The system should support: - OpenClaw gateway (primary) - Direct OpenAI/Anthropic API keys - Local models via Ollama or similar

The LLM is called via a clean interface:

```
class LLMBackend:
    def elaborate(self, spec_text: str, guidance: str, section: str | None = None) -> ElaborationResult
    def compile(self, elaborated_spec: str, test_spec: str, guidance: str) -> CompilationResult
```

5.2 Existing Validation as Quality Gate

The current SpecAtom-HS pipeline runs after both elaboration and compilation as a quality gate. If the LLM produces output that fails validation (e.g., broken concept references, missing coverage), the system reports the failures and asks the LLM to fix them (automatic retry, configurable max retries).

5.3 Sandboxed Test Execution

Tests run in a sandboxed environment (container, venv, or subprocess with resource limits). The system never executes LLM-generated code with access to the host filesystem, network, or secrets. Test execution is opt-in — the user clicks “Run Tests” explicitly.

5.4 Artifact Storage

Each spec project gets a directory:

```
projects/  
  my-spec-2026-08-06T12:00:00/  
    original.plain  
    elaborated.plain  
    tests.plain  
    elaborated.reviewed.plain  
    tests.reviewed.plain  
    logical-skeleton.metta  
    logical-review.json  
    compiled.metta  
  modules/  
    data_processing.py  
    model.py  
  tests.metta  
  tests.py  
  traceability.json  
  test-results.json  
  logs/  
    elaboration-log.json  
    review-log.json  
    compilation-log.json
```

6. Implementation Plan

Step 0: Preserve the Existing Foundation

Effort: minimal **What:** Rename `POST /api/compile` to `POST /api/validate`. Ensure existing tests pass. Tag the current codebase as `v0.1-validation-only`. **Verification:** All existing tests pass. The web UI still works with the renamed endpoint.

Step 1: Artifact Storage & Project Model

Effort: small **What:** Implement the project directory structure (§5.4). Add a `Project` dataclass that tracks the state of a spec through the pipeline phases. Add `POST /api/projects` (create), `GET /api/projects` (list), `GET /api/projects/:id` (status). **Verification:** Can create a project from a `.plain` file, retrieve its status, and list all projects.

Step 2: LLM Backend Abstraction

Effort: medium **What:** Implement the `LLMBackend` interface (§5.1). Start with OpenClaw gateway support. Add configuration for model selection, temperature, max tokens. Add the elaboration-log and compilation-log JSON formats. **Verification:** Can send a prompt to the configured LLM and receive a response. Logs are written correctly.

Step 3: Elaboration Phase

Effort: large **What:** Implement `POST /api/elaborate`. The endpoint accepts a `.plain` spec and an optional guidance prompt. It calls the LLM to produce an elaborated spec and an elaborated test spec. It then runs the existing SpecAtom-HS validation pipeline on the elaborated output. If validation fails, it retries with the failures included in the prompt (up to configurable max retries). Store all artifacts in the project directory. **Key prompts to engineer:** - System prompt defining the elaboration task, output format, and constraints - Example few-shot elaborations (use the existing `auth_service.plain` and `ml_timeseries.plain` as inputs, write gold-standard elaborations as reference outputs) - Retry prompt that includes validation failures **Verification:** Given `ml_timeseries.plain`, the elaboration produces a detailed spec covering train/test split, normalization, hyperparameter selection, evaluation metrics, and reproducibility. The elaborated test spec includes at least one test per functional requirement. Validation passes on the elaborated output.

Step 4: Review Workflow

Effort: medium **What:** Implement the review interface in the web UI (§4.2.4) and the `POST /api/review/:id` endpoint. Support approve/reject/edit per section. Support re-elaboration of individual sections. Persist review state. **Verification:** Can approve an elaborated spec through the UI. Can edit a section and re-elaborate it. Review decisions are persisted and retrievable.

Step 5: Logical IR / Verification Skeleton

Effort: large **What:** Add an `emit_logical_skeleton` sibling to the existing reified-atom emitter. It converts approved elaborated clauses into typed declarations, contracts, invariants, information-flow relations, test obligations, and explicit operational holes. Implement deterministic checks where possible and a constrained LLM/formal-reasoner review adapter for semantic findings. Add `POST /api/logical-ir/:id`, a review gate, and the Logical IR/Logical Review UI tabs. **Verification:** A manually authored gold skeleton for `auth_service.plain` and an elaborated `ml_timeseries` skeleton both: retain complete clause provenance; expose each ML operation as a contract plus an explicit grounding gap; report missing/contradictory definitions; and prevent compilation while a critical finding remains unresolved.

Step 6: Compilation Phase

Effort: large **What:** Implement `POST /api/compile/:id`. The endpoint takes an approved elaborated spec + test spec + logical-IR approval + optional compilation guidance prompt and calls the LLM to produce literate MeTTa, grounded Python modules, and executable tests. Run the SpecAtom-HS validation pipeline on the compiled MeTTa.

Generate the traceability mapping. **Key prompts to engineer:** - System prompt defining the compilation task, MeTTa coding conventions, Python grounding patterns, test structure - Examples of well-compiled specs (write gold-standard compiled output for the `auth_service` elaboration) - Traceability extraction prompt or post-processing logic **Verification:** Given an approved elaborated spec, the compilation produces syntactically valid MeTTa and Python. The traceability JSON maps every `[id:...]` to at least one code location and one test. Validation passes.

Step 7: Test Execution

Effort: medium **What:** Implement sandboxed test execution (§5.3). Add `POST /api/test/:id` and `GET /api/test/:id`. Run pytest on the generated Python tests. Parse results into structured JSON. Add the Test Results tab to the web UI (§4.2.6). **Verification:** Can execute generated tests in a sandboxed environment. Results are displayed in the web UI with per-test pass/fail status and drill-down to output.

Step 8: Workflow UI

Effort: medium **What:** Add the workflow progress bar (§4.2.1), guidance prompt panel (§4.2.2), compilation output panel (§4.2.5), and connect all phases in the UI. Each phase transitions to the next via explicit user action. **Verification:** Can walk through the entire pipeline in the web UI: paste spec → elaborate → review → compile → test → see report.

Step 9: Traceability Report

Effort: small **What:** Implement the Report phase (§3.7). Generate the full provenance chain view. Add spec-item coverage visualization to the test results panel. **Verification:** The report shows which spec items are covered by passing tests, which have failures, and which lack coverage.

Step 9: Hardening & Polish

Effort: medium **What:** - Error handling for LLM failures, timeouts, malformed outputs - Rate limiting for LLM API calls - Progress indicators for long-running elaboration/compilation - Responsive UI for mobile - Keyboard shortcuts for review workflow - Export project as zip **Verification:** The system handles LLM failures gracefully. The UI is responsive and usable.

Step 10 (optional): Version History

Effort: medium **What:** Implement the version history sidebar (§4.2.7). Each phase run creates a versioned snapshot. Support diff and rollback. **Verification:** Can view version history, diff between versions, and roll back to a previous version.

7. What This Does NOT Cover (Explicit Non-Goals for v2)

- Full Plain grammar parsing (the existing conservative parser is sufficient)
 - Automatic deployment or CI/CD integration
 - Multi-user collaboration (single-user workflow for now)
 - Real-time collaborative editing
 - Integration with external issue trackers or project management tools
 - Automatic publication of generated code to repositories
 - Write-back from generated code to spec (spec is source of truth, code is derived)
-

8. Success Criteria

The system is successful when:

1. A domain expert can write a 10-line sketchy spec, click “Elaborate,” read the detailed output, confirm it matches their intent (or correct it), click “Compile,” and receive working code with tests — all within the web UI.
 2. The generated tests actually test what the spec says — not token-level matching, but behavioral verification that a knowledgeable reviewer would agree covers the requirement.
 3. The traceability chain is unbroken: for any test failure, the user can trace back to the exact spec item and the exact elaborated description that the failing code was supposed to implement.
 4. The existing validation infrastructure (source provenance, concept consistency, coverage obligations) catches errors that the LLM introduces, before the user ever sees them.
-

9. Open Questions for Review

1. **MeTTa as primary executable vs. Python with MeTTa annotations?** The spec above assumes MeTTa is the primary executable with Python grounded in. If most real specs will produce primarily Python code with MeTTa as the type/reasoning layer, the architecture should reflect that. What does Ben prefer?

2. **Scope of “guidance prompt.”** Should the guidance prompt be structured (key-value fields: target language, libraries, constraints) or free-text? Free-text is more flexible; structured is more predictable.
3. **AI advisory committee for review.** Should Phase 3 support automated review by multiple LLMs (e.g., one elaborates, another reviews the elaboration for consistency/completeness)? This is architecturally simple but has cost implications.
4. **Integration with Hyperon.** Should the compiled MeTTa be loadable into a Hyperon Atomspace directly? If so, we need to define the grounding conventions for Python modules.
5. **Spec versioning granularity.** Should every edit create a new version, or only explicit “save” actions?

This document should be reviewed by ProtoCosmoBot, Hugo, and Ben before implementation begins. The implementation plan (§6) is designed so that each step produces a testable increment — no step depends on all prior steps being perfect.